

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL APLIKASI WEB DAN MOBILE MANUAL PRAKTIKUM –
COMPONENT, NAVIGATION, DAN STYLING DI REACT NATIVE +
PROYEK MOBILE**

Nama Mahasiswa : Wianda Sukandari
NIM : 23051430002
Kelas : A
Tanggal Praktikum : 29 May 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web Dan Mobile Manual Praktikum (Pertemuan 15)
Topik Utama	Component, Navigation, dan Styling di React Native + Proyek Mobile
Sub-Topik	React Navigation (Stack Navigator), Passing Data Antar Layar, dan Proyek Inventori Sederhana.
Durasi Praktikum	340 Menit
Tanggal Praktikum	29 Mei 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep navigasi layar di aplikasi mobile (Stack).
2	Mampu menginstall dan mengonfigurasi `react-navigation`
3	Membuat aplikasi multi-halaman: Beranda (List) → Detail Item.
4	Meneruskan data (params) dari satu layar ke layar lain.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)

4	Expo CLI / Expo Go	Versi terbaru	Menjalankan aplikasi React Native
5	React Navigation & React Native Picker	Versi terbaru	Navigasi antar halaman dan dropdown status inspeksi

D. DASAR TEORI

D.1 Konsep Utama

Pada praktikum ini dipelajari konsep navigasi pada aplikasi mobile menggunakan React Navigation, khususnya Stack Navigator. Stack Navigator bekerja seperti tumpukan halaman, di mana halaman baru akan ditambahkan di atas halaman sebelumnya. Ketika pengguna berpindah ke halaman lain, halaman sebelumnya tetap tersimpan sehingga pengguna dapat kembali ke halaman sebelumnya menggunakan tombol back.

Selain navigasi, praktikum ini juga membahas proses pengiriman data antar layar (passing parameter). Data dari halaman daftar inventori dapat dikirim ke halaman detail melalui parameter navigasi. Dengan cara ini, informasi yang dipilih pengguna pada halaman utama dapat ditampilkan secara lengkap pada halaman detail tanpa perlu membuat data baru.

Konsep lain yang dipelajari adalah pembuatan aplikasi multi-halaman sederhana menggunakan React Native. Aplikasi terdiri dari halaman daftar item dan halaman detail item yang saling terhubung melalui navigasi. Penerapan konsep ini membantu dalam memahami struktur aplikasi mobile yang lebih terorganisir dan mudah dikembangkan untuk kebutuhan yang lebih kompleks.

D.2 Keterkaitan dengan Teknik Industri

Materi navigasi aplikasi mobile memiliki keterkaitan dengan bidang Teknik Industri karena banyak sistem informasi industri saat ini berbasis aplikasi mobile. Perusahaan membutuhkan aplikasi yang mampu menampilkan data secara terstruktur sehingga pengguna dapat berpindah dari satu halaman ke halaman lainnya dengan mudah dan cepat.

Sebagai contoh, pada sistem inventori gudang, operator dapat melihat daftar barang pada halaman utama kemudian memilih salah satu barang untuk melihat detail stok, lokasi

penyimpanan, dan informasi lainnya. Proses perpindahan dari daftar barang ke detail barang memanfaatkan konsep navigasi yang dipelajari dalam praktikum ini.

Selain itu, aplikasi monitoring produksi, maintenance mesin, maupun manajemen logistik juga menggunakan konsep yang sama. Teknisi atau supervisor dapat memilih data tertentu pada halaman utama dan melihat informasi detail pada halaman berikutnya. Oleh karena itu, pemahaman mengenai navigasi dan pengiriman data antar layar sangat penting dalam pengembangan sistem informasi yang mendukung kegiatan operasional di dunia industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

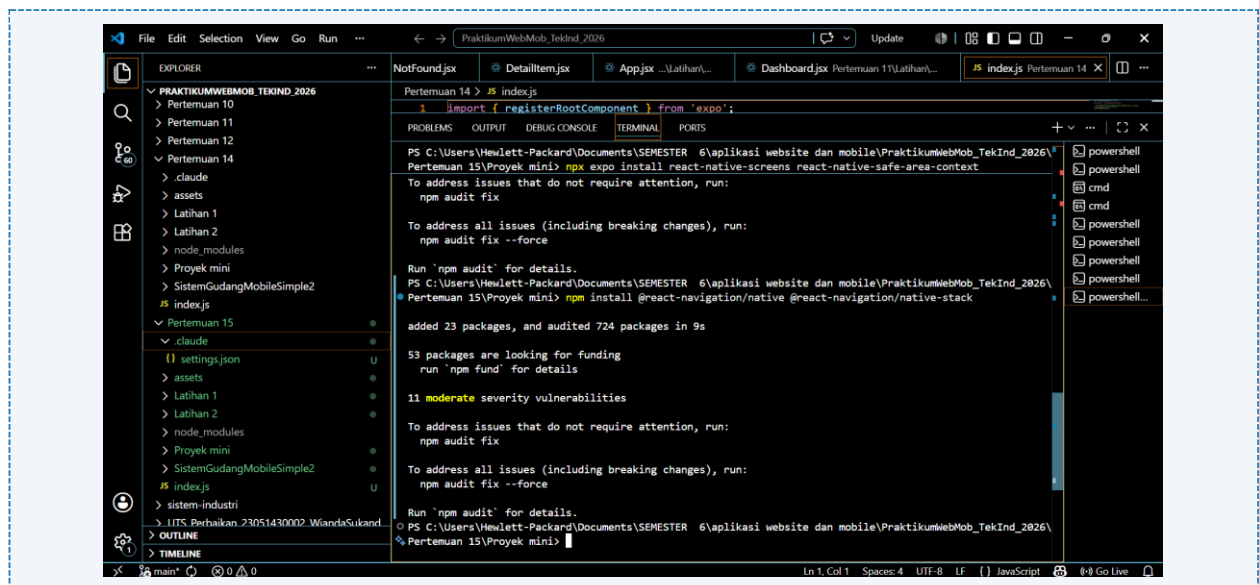
E.1 Langkah 1 – Struktur Folder

Pertama siapin project Expo di pertemuan 14 dan copy ke pertemuan 15. Terus install library dan dependensinya (kode dibawah). Kemudian buat folder baru di dalam project bernama `screens`. Di dalamnya buat dua file:

1. `HomeScreen.js` (Daftar Inventori)
2. `DetailScreen.js` (Detail Barang)

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */  
npm install @react-navigation/native @react-navigation/native-stack npx  
expo install react-native-screens react-native-safe-area-context
```

Screenshot Hasil:



Gambar E.1 – Bukti install

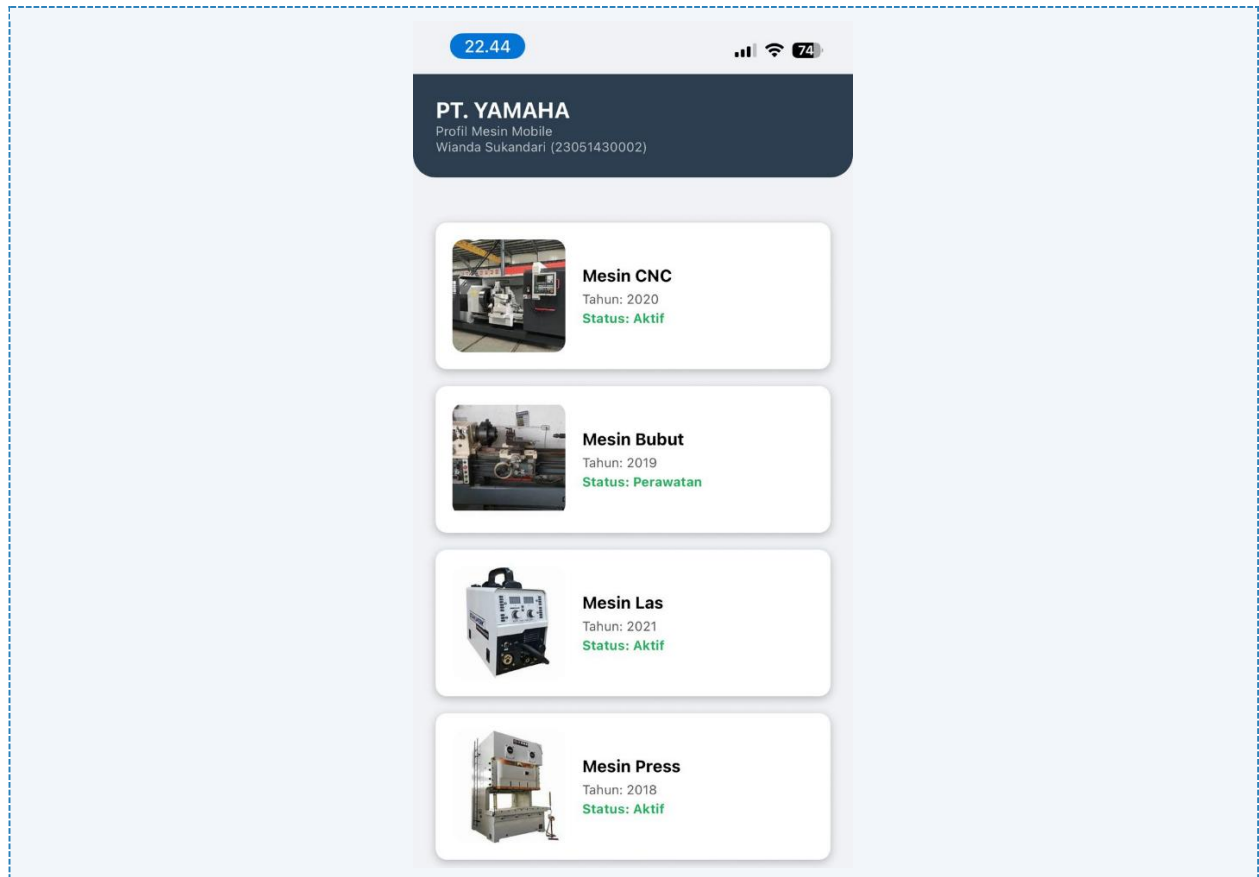
E.2 Langkah 2 – Membuat HomeScreen (Daftar Inventori)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet } from 'react-native';
// Data Mock Inventori
const DATA_INVENTORI = [
  { id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1' },
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-3' },
  { id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang Luar' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' }, // Stok Habis
];
function HomeScreen({ navigation }) {
  // Fungsi Render Item untuk FlatList
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() => navigation.navigate('Detail', { itemData: item })}
    >
      <Text style={styles.itemTitle}>{item.nama}</Text>
      <View style={styles.itemInfo}>
        <Text style={styles.itemSub}>Stok: {item.stok}</Text>
        <Text style={styles.itemSub}>{item.lokasi}</Text>
      </View>
    </TouchableOpacity>
  );
  return (
    <View style={styles.container}>
      <Text style={styles.header}>Daftar Inventori Gudang</Text>
      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={item => item.id}
      />
    </View>
  );
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    paddingTop: 20,
  },
  header: {
    fontSize: 22,
    fontWeight: 'bold',
    marginBottom: 15,
    paddingHorizontal: 15,
  },
});
```

```
    itemContainer: {
      backgroundColor: '#f9f9f9',
      padding: 15,
      marginVertical: 8,
      marginHorizontal: 15,
      borderRadius: 8,
      borderWidth: 1,
      borderColor: '#ddd',
    },
    itemTitle: {
      fontSize: 18,
      fontWeight: 'bold',
      color: '#333',
    },
    itemInfo: {
      flexDirection: 'row',
      justifyContent: 'space-between',
      marginTop: 5,
    },
    itemSub: {
      color: '#666',
    },
  });
export default HomeScreen;
```

Screenshot Hasil:

Tampilannya masih sama kaya project 14 karena file saya pakai hasil terakhir (mini proyek) di pertemuan 14.



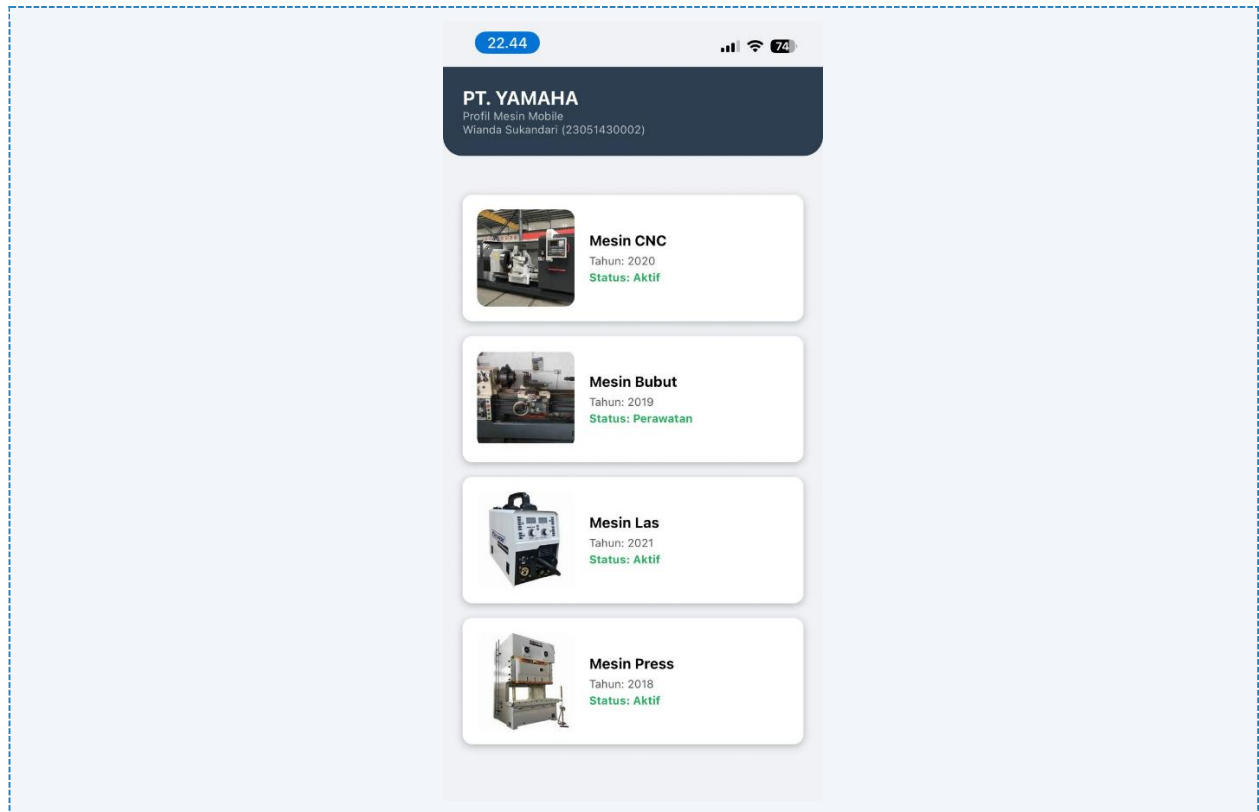
Gambar E.2 – Membuat HomeScreen (Daftar Inventori)

E.3 Langkah 3 – Membuat DetailScreen

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React from 'react';
import { View, Text, StyleSheet, Button } from 'react-native';
function DetailScreen({ route, navigation }) {
  // Menerima data yang dikirim dari HomeScreen
  const { itemData } = route.params;
  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>
        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text style={[styles.value, itemData.stok < 20 ?
styles.dangerText :
styles.successText]}>
          {itemData.stok} Unit
        </Text>
        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>
      </View>
    </View>
  );
}
```

```
        <Button title="Kembali ke Daftar" onPress={() =>
navigation.goBack()} />
      </View>
    );
  }
  const styles = StyleSheet.create({
    container: {
      flex: 1,
      justifyContent: 'center',
      padding: 20,
      backgroundColor: '#f0f0f0',
    },
    card: {
      backgroundColor: 'white',
      padding: 20,
      borderRadius: 10,
      marginBottom: 20,
      elevation: 3,
    },
    label: {
      fontSize: 14,
      color: '#7f8c8d',
      marginTop: 10,
    },
    value: {
      fontSize: 20,
      fontWeight: 'bold',
      color: '#2c3e50',
    },
    dangerText: {
      color: 'red',
    },
    successText: {
      color: 'green',
    }
  });
  export default DetailScreen;
```

Screenshot Hasil:



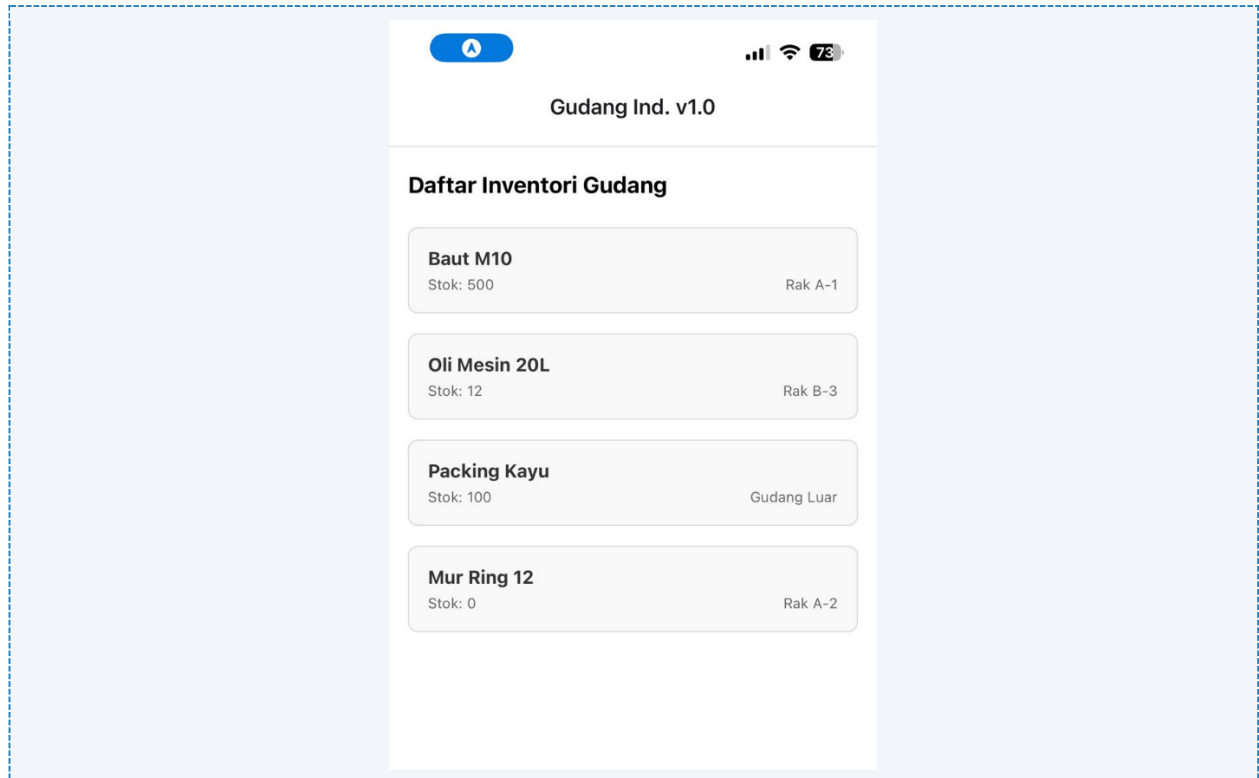
Gambar E.3 – Membuat DetailScreen

E.4 Langkah 4 – Menghubungkan Navigation di App.js

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
// Import Screens
import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';
// Membuka Stack Navigator
const Stack = createNativeStackNavigator();
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{ title: 'Gudang Ind. v1.0' }}
        />
        <Stack.Screen
          name="Detail"
          component={DetailScreen}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

```
options={({ route }) => ({ title: route.params?.itemData.nama ||  
'Detail' })}  
/>  
</Stack.Navigator>  
</NavigationContainer>  
);  
}
```

Screenshot Hasil:



Gambar E.4 – Menghubungkan Navigation di App.js

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `DetailScreen`, jika stok = 0, munculkan tombol "Request Stok Darurat" (gunakan Button tambahan).

Jawaban / Kode Solusi:

```
import React from 'react';
```

```
import { View, Text, StyleSheet, Button, Alert } from 'react-native';

function DetailScreen({ route, navigation }) {
  // Menerima data yang dikirim dari HomeScreen
  const { itemData } = route.params;

  const handleRequestStok = () => {
    Alert.alert(
      'Request Stok Darurat',
      'Permintaan stok darurat berhasil dikirim!'
    );
  };

  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>

        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text
          style={[
            styles.value,
            itemData.stok < 20
              ? styles.dangerText
              : styles.successText,
          ]>
          >
            {itemData.stok} Unit
        </Text>

        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>

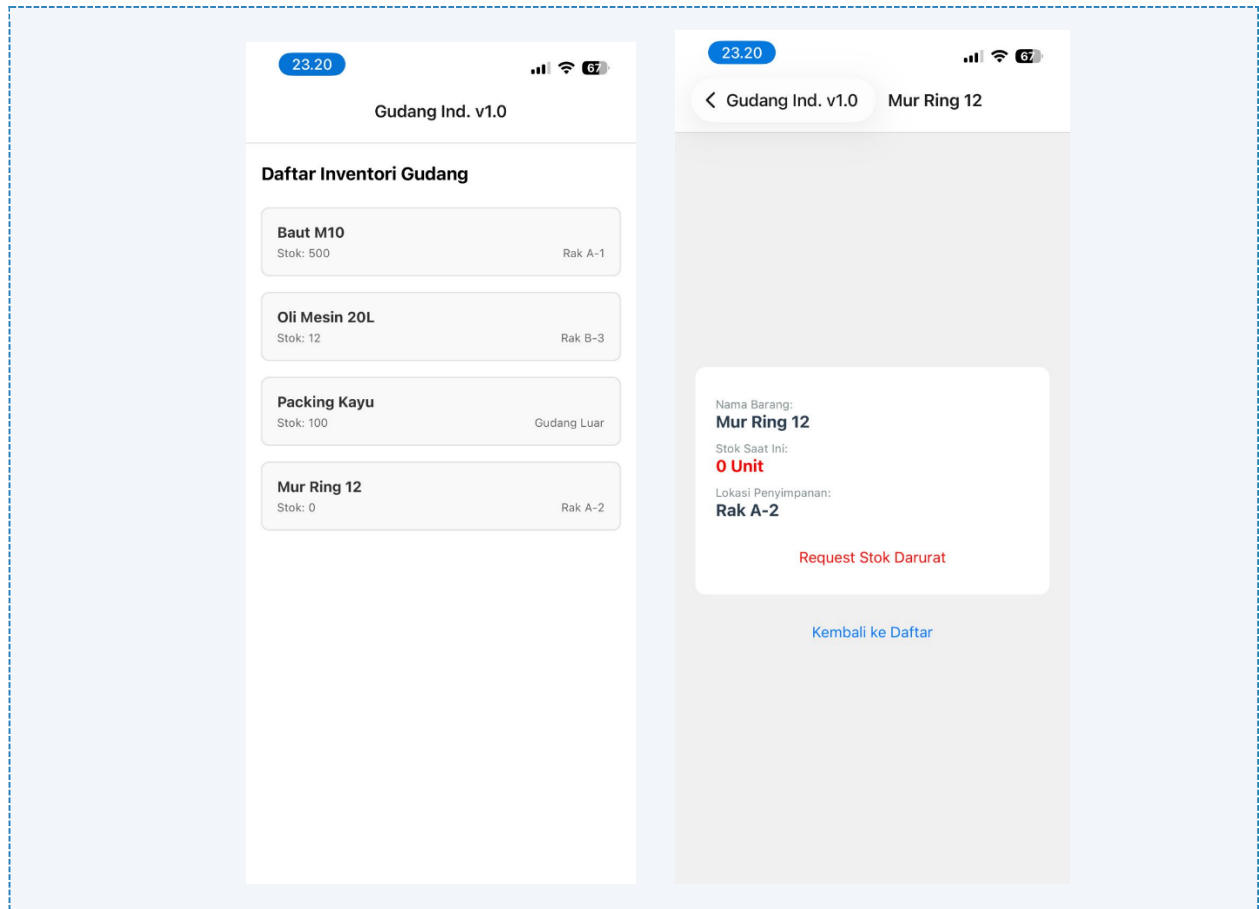
        { /* Conditional Rendering */ }
        {itemData.stok === 0 && (
          <View style={styles.buttonContainer}>
            <Button
              title="Request Stok Darurat"
              color="red"
              onPress={handleRequestStok}
            />
          </View>
        ) }
      </View>

      <Button
        title="Kembali ke Daftar"
        onPress={() => navigation.goBack()}
      />
    </View>
  );
}
```

```
        </View>
      );
    }

    const styles = StyleSheet.create({
      container: {
        flex: 1,
        justifyContent: 'center',
        padding: 20,
        backgroundColor: '#f0f0f0',
      },
      card: {
        backgroundColor: 'white',
        padding: 20,
        borderRadius: 10,
        marginBottom: 20,
        elevation: 3,
      },
      label: {
        fontSize: 14,
        color: '#7f8c8d',
        marginTop: 10,
      },
      value: {
        fontSize: 20,
        fontWeight: 'bold',
        color: '#2c3e50',
      },
      dangerText: {
        color: 'red',
      },
      successText: {
        color: 'green',
      },
      buttonContainer: {
        marginTop: 20,
      },
    });

    export default DetailScreen;
```



Gambar F.1 – Conditional Rendering

F.2 Latihan 2 – Passing Data

Pertanyaan / Instruksi Latihan:

Di `HomeScreen`, tambahkan tombol FAB (Floating Action Button) atau Button di bawah untuk "Tambah Barang". Saat ditekan, navigasi ke layar (screen baru bernama `TambahScreen`) yang berisi form kosong.

Jawaban / Kode Solusi:

Buat file baru didalam folder screen yaitu TambahScreen.js dan ubah kode di HomeScreen.js dan App.js

```
import React from 'react';
import { View, Text, TextInput, StyleSheet, Button } from 'react-native';

function TambahScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Form Tambah Barang</Text>

      <TextInput
        style={styles.input}

```

```
        placeholder="Nama Barang"
      />

      <TextInput
        style={styles.input}
        placeholder="Jumlah Stok"
        keyboardType="numeric"
      />

      <TextInput
        style={styles.input}
        placeholder="Lokasi Penyimpanan"
      />

      <Button
        title="Simpan"
        onPress={() => alert('Data berhasil disimpan')}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#fff',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    padding: 10,
    marginBottom: 15,
  },
});

export default TambahScreen;

import React from 'react';
import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
```

```
    StyleSheet,
    Button
  } from 'react-native';

const DATA_INVENTORI = [
  { id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1' },
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-3' },
  { id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang Luar' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' },
];

function HomeScreen({ navigation }) {

  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() =>
        navigation.navigate('Detail', {
          itemData: item
        })
      }
    >
    <Text style={styles.itemTitle}>{item.nama}</Text>

    <View style={styles.itemInfo}>
      <Text style={styles.itemSub}>
        Stok: {item.stok}
      </Text>

      <Text style={styles.itemSub}>
        {item.lokasi}
      </Text>
    </View>
    </TouchableOpacity>
  );

  return (
    <View style={styles.container}>
      <Text style={styles.header}>
        Daftar Inventori Gudang
      </Text>

      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={(item) => item.id}
      />

      <View style={styles.buttonContainer}>
        <Button
```

```
        title="Tambah Barang"
        onPress={() =>
            navigation.navigate('Tambah')
        }
    />
</View>
</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    paddingTop: 20,
  },
  header: {
    fontSize: 22,
    fontWeight: 'bold',
    marginBottom: 15,
    paddingHorizontal: 15,
  },
  itemContainer: {
    backgroundColor: '#f9f9f9',
    padding: 15,
    marginVertical: 8,
    marginHorizontal: 15,
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#ddd',
  },
  itemTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#333',
  },
  itemInfo: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 5,
  },
  itemSub: {
    color: '#666',
  },
  buttonContainer: {
    margin: 15,
  },
});

export default HomeScreen;
```

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';
import TambahScreen from './screens/TambahScreen';

const Stack = createNativeStackNavigator();

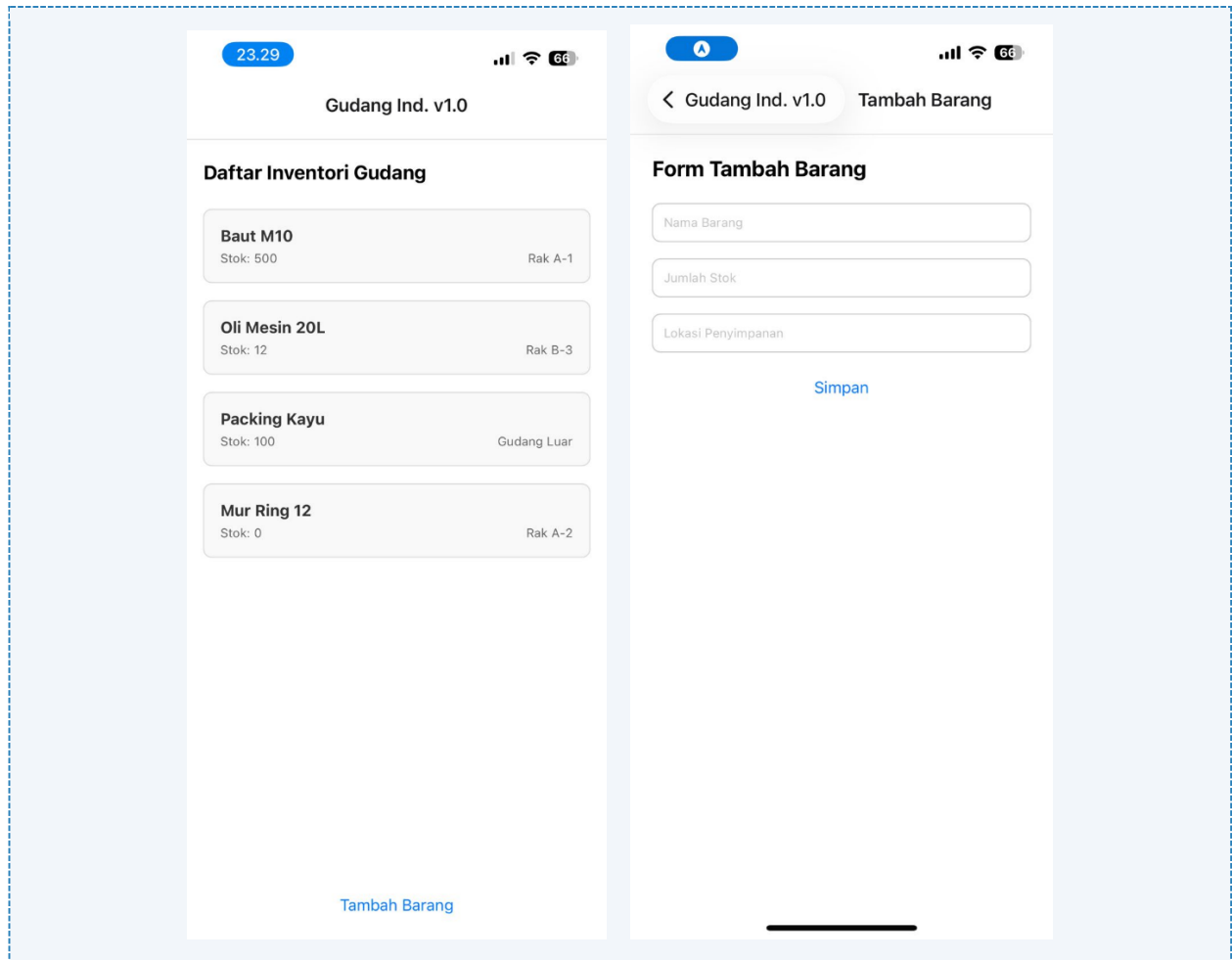
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">

        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{ title: 'Gudang Ind. v1.0' }}
        />

        <Stack.Screen
          name="Detail"
          component={DetailScreen}
          options={({ route }) => ({
            title:
              route.params?.itemData.nama ||
              'Detail'
          })}
        />

        <Stack.Screen
          name="Tambah"
          component={TambahScreen}
          options={{
            title: 'Tambah Barang'
          }}
        />

      </Stack.Navigator>
    </NavigationContainer>
  );
}
```



Gambar F.2 – Passing Data

F.3 Tugas Proyek Mini – Aplikasi Inspeksi QC

Deskripsi proyek mini:

Buat aplikasi inspeksi sederhana.

- Home: Daftar item yang perlu diinspeksi.
- Detail: Foto item, standar kualitas, dan dropdown status (Lolos/Gagal).
- Simulasi: Saat user memilih "Gagal" di layar detail dan kembali, ubah warna teks di Home menjadi merah (memerlukan sedikit logika state management sederhana).

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, aplikasi inspeksi QC dibuat menggunakan React Native dengan konsep navigasi antar layar (Stack Navigation). Aplikasi terdiri dari halaman Home yang menampilkan daftar item yang akan diperiksa, serta halaman Detail yang digunakan untuk melihat informasi item secara lebih lengkap. Data item disimpan dalam state sehingga dapat diperbarui ketika pengguna melakukan inspeksi.

Setiap item inspeksi dilengkapi dengan nama komponen, foto item, standar kualitas, dan pilihan status inspeksi menggunakan dropdown (Picker). Pengguna dapat memilih status "Lolos" atau "Gagal" sesuai hasil pemeriksaan. Setelah status dipilih dan disimpan, data item akan diperbarui sehingga hasil inspeksi dapat ditampilkan kembali pada halaman utama.

Untuk mensimulasikan proses Quality Control di industri, diterapkan logika state management sederhana. Jika suatu item diberi status "Gagal", maka warna teks item tersebut pada halaman Home akan berubah menjadi merah sebagai indikator adanya ketidaksesuaian kualitas. Pendekatan ini menunjukkan bagaimana data dapat dikirim antar layar, diolah, dan ditampilkan kembali secara dinamis untuk mendukung proses monitoring kualitas produk.

Kode Utama:

```
import React, { useState } from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';

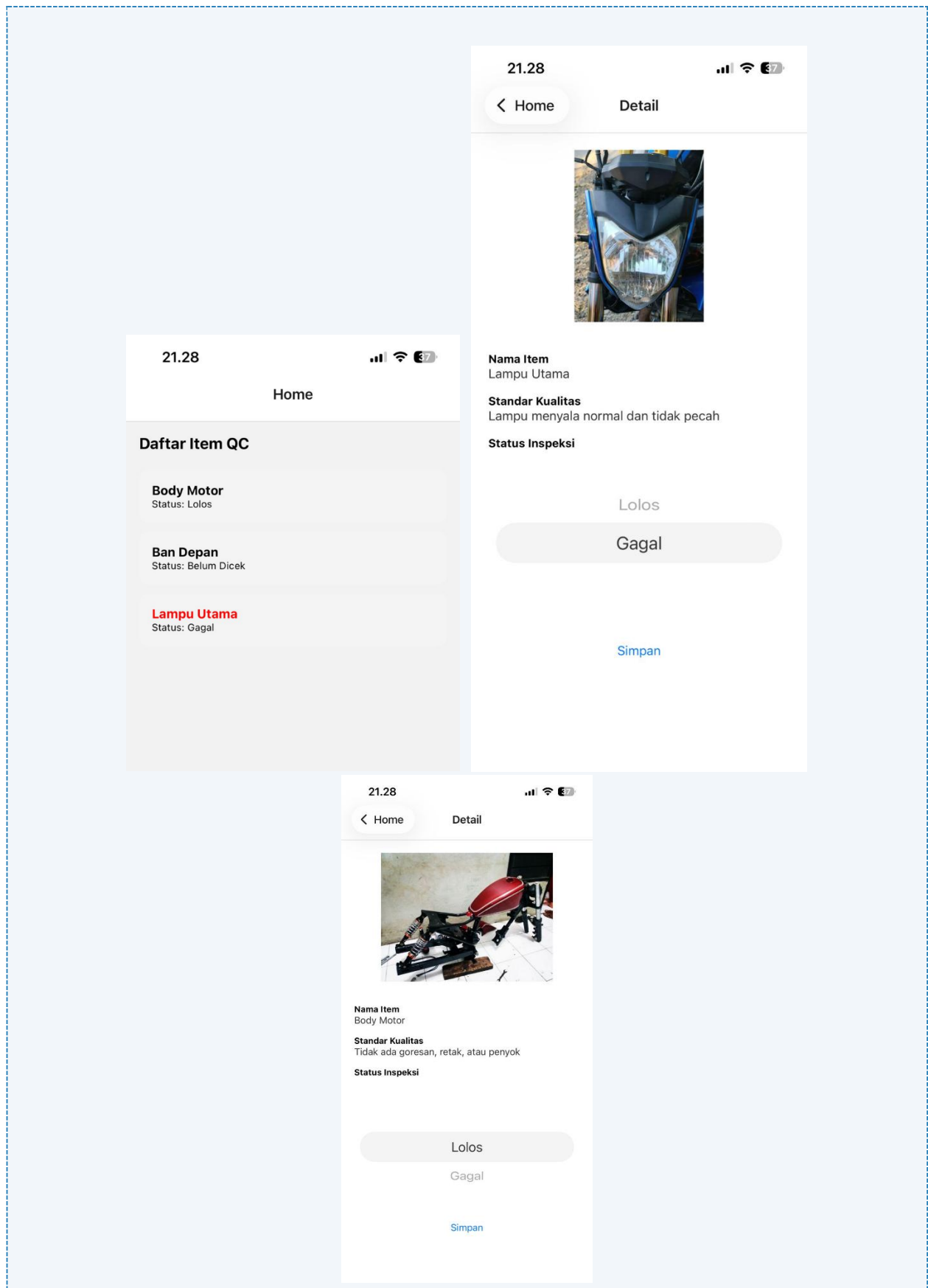
const Stack = createNativeStackNavigator();

export default function App() {
  const [items, setItems] = useState([
    {
      id: '1',
      nama: 'Body Motor',
      standar: 'Tidak ada goresan, retak, atau penyok',
      gambar: require('./assets/body_motor.jpg'),
      status: 'Belum Dicek',
    },
    {
      id: '2',
      nama: 'Ban Depan',
      standar: 'Tekanan sesuai standar dan tidak aus',
      gambar: require('./assets/ban_depan.jpg'),
      status: 'Belum Dicek',
    },
    {
      id: '3',
```

```
    nama: 'Lampu Utama',
    standar: 'Lampu menyala normal dan tidak pecah',
    gambar: require('./assets/lampu_utama.jpg'),
    status: 'Belum Dicek',
  },
]);

return (
  <NavigationContainer>
    <Stack.Navigator>
      <Stack.Screen name="Home">
        { (props) => (
          <HomeScreen
            {...props}
            items={items}
          />
        ) }
      </Stack.Screen>

      <Stack.Screen name="Detail">
        { (props) => (
          <DetailScreen
            {...props}
            items={items}
            setItems={setItems}
          />
        ) }
      </Stack.Screen>
    </Stack.Navigator>
  </NavigationContainer>
);
}
```



Gambar F.3 – Output Aplikasi Inspeksi QC

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh telah sesuai dengan teori React Native Navigation dan state management sederhana yang dipelajari pada modul ini. Aplikasi berhasil menerapkan konsep Stack Navigator sehingga pengguna dapat berpindah dari halaman Home menuju halaman Detail dan kembali ke halaman sebelumnya. Selain itu, data item inspeksi dapat dikirim dari HomeScreen ke DetailScreen menggunakan parameter navigasi (passing data), sehingga informasi item yang dipilih dapat ditampilkan secara spesifik pada halaman detail.

Pada halaman DetailScreen, konsep conditional rendering dan state management juga berhasil diterapkan melalui komponen Picker untuk memilih status inspeksi "Lolos" atau "Gagal". Status yang dipilih pengguna kemudian disimpan ke state aplikasi menggunakan useState dan diteruskan kembali ke halaman Home. Mekanisme ini menunjukkan bahwa perubahan data dapat dikelola dan ditampilkan secara dinamis sesuai interaksi pengguna.

Hasil output aplikasi menunjukkan bahwa ketika suatu item diberi status "Gagal", warna teks item pada HomeScreen berubah menjadi merah. Perubahan tampilan tersebut membuktikan bahwa state management sederhana berjalan dengan baik karena data yang diperbarui pada DetailScreen dapat memengaruhi tampilan komponen lain. Dengan demikian, seluruh fitur yang diminta dalam proyek mini berhasil diimplementasikan dan sesuai dengan konsep teori yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Error "Unable to resolve ../assets/images/body_motor.jpg" karena file gambar tidak ditemukan pada folder yang ditentukan.	Membuat folder assets/images dan memastikan nama file gambar sesuai dengan yang dipanggil menggunakan require().
Gambar tidak muncul pada aplikasi meskipun tidak ada error.	Memastikan penggunaan komponen Image sudah benar dan path gambar sesuai dengan lokasi penyimpanan file.
Error karena deklarasi komponen Image dilakukan dua kali pada DetailScreen.	Menghapus import yang duplikat dan menggabungkan seluruh komponen React Native dalam satu baris import.

G.3 Kaitan dengan Penerapan di Industri

Materi React Native Navigation dan state management memiliki keterkaitan yang erat dengan kebutuhan industri, khususnya dalam pengembangan aplikasi mobile untuk mendukung proses operasional perusahaan. Kemampuan membuat navigasi antar halaman memungkinkan pengembang menyusun alur kerja aplikasi yang terstruktur dan mudah digunakan oleh pengguna.

Dalam bidang Teknik Industri, aplikasi serupa dapat digunakan untuk mendukung proses inspeksi kualitas produk di area produksi. Petugas Quality Control dapat melihat daftar item yang harus diperiksa, membuka detail setiap item, melihat standar kualitas yang berlaku, kemudian memberikan status lulus atau gagal secara langsung melalui aplikasi mobile. Data inspeksi tersebut dapat digunakan untuk pengambilan keputusan dan monitoring kualitas secara real-time.

Contoh penerapan lainnya adalah pada sistem inspeksi kendaraan, audit gudang, pemeriksaan peralatan produksi, maupun maintenance mesin. Dengan adanya aplikasi mobile berbasis React Native, proses pencatatan hasil inspeksi menjadi lebih cepat, akurat, dan terdokumentasi dengan baik dibandingkan metode pencatatan manual menggunakan kertas.

H. KESIMPULAN

1	Praktikum berhasil menerapkan navigasi antar halaman menggunakan React Navigation dengan metode Stack Navigator sehingga pengguna dapat berpindah dari halaman daftar inspeksi ke halaman detail item.
2	Data item inspeksi dapat dikirim dari HomeScreen ke DetailScreen menggunakan teknik passing data melalui parameter navigasi, sehingga informasi setiap item dapat ditampilkan secara spesifik.
3	Aplikasi berhasil menampilkan gambar komponen yang diinspeksi, standar kualitas, serta fitur pemilihan status inspeksi menggunakan komponen Picker.
4	Konsep state management sederhana menggunakan useState berhasil diterapkan untuk menyimpan dan memperbarui status inspeksi setiap item secara dinamis.
5	Simulasi perubahan status inspeksi berhasil dilakukan, dimana item yang berstatus "Gagal" ditampilkan dengan warna merah pada halaman Home sehingga memudahkan identifikasi item yang tidak memenuhi standar kualitas.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React Native Documentation . (2026). <i>Getting Started with React Native</i> . React Native.
[3]	React Navigation Documentation . (2026). <i>Getting Started</i> . React Navigation.
[4]	Expo Documentation . (2026). <i>Expo Documentation</i> . Expo.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Wianda Sukandari NIM: 23051430002	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm Bawah: 3 cm Kiri: 4 cm Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → ... → Laporan Modul 12 → Laporan Modul 14 & 15.
c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```

PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)
  
```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS

— Selamat mengerjakan —